

**Codebook
of
BU_Optimizers
University of Barishal**


```
dis.assign(n+1, LLONG_MAX);

dis[source] = 0;

for(int i = 1; i <= n; i++) {
    bool updated = false;
    for(auto &[u,v,w] : edges) {
        if(dis[u] != LLONG_MAX &&
           dis[u]+w < dis[v]) {
            dis[v] = dis[u]+w;
            updated = true;
        }
        if(i == n) hasNegativeCycle = true;
    }
    if(!updated) break;
} };
```

5. Binary Lifting

Time Complexity: $O(n \cdot \log(n))$ (create)
other function : $O(\log(n))$

```
class Binary_lifting{
public :

    vector<vector<ll>> up;
    vector<ll> depth;

    Binary_lifting(ll n ,
        vector<vector<ll>> &edge){
        up.resize(n+1,vector<ll>(20,-1));
        depth.resize(n+1,0);

        dfs(1,-1,edge);

        for (ll i = 1; i < 20; i++) {
            for (ll j = 1; j <= n; j++){
                if(up[j][i-1] != -1) up[j][i] = up[
                    up[j][i-1] ][i-1];
            }
        }
    }
};
```

```
void dfs(ll node , ll parent ,
    vector<vector<ll>> &edge ){
    up[node][0] = parent;
    if(parent != -1) depth[node] =
        depth[parent] + 1;

    for(auto child:edge[node]){
        if(child != parent)
            dfs(child,node,edge);
    }
}

ll k_ancestor(ll node , ll k){
    while( k != 0){
        ll x = log2(k);
        node = up[node][x];
        k -= (1LL << x);
        if(node == -1 ) return -1;
    }
    return node;
}

ll lca(ll a , ll b){
    if(depth[a] < depth[b]) swap(a,b);
    ll diff = depth[a] - depth[b];
    a = k_ancestor(a,diff);

    if(a == b) return a;

    for (ll i = 19; i >= 0 ; i--){
        if(up[a][i] != -1 && up[b][i] !=
            up[a][i]){
            a = up[a][i];
            b = up[b][i];
        }
    }

    return up[a][0];
}

ll distance(ll a , ll b){
    return depth[a] + depth[b] - (2 *
        depth[lca(a,b)]);
}
};
```

6. Strongly Connected Components (Kosaraju's algo)

```
// Time complexcity:  $O(n + m \log m)$ 
// Found the number of SCC
// Unique SCC Id for each node
// Make DAG with SCC
class SCC{

public :

    vector<ll> vis;
    stack<ll> finished;
    vector<vector<ll>> reverse_edge;
    ll number_of_scc = 0;
    vector<ll> scc_id;
    vector<vector<ll>> scc_graph;

    SCC( ll n , ll start_node , ll
        end_node, vector<vector<ll>>
        &edge ){
        vis.resize(n+1);
        reverse_edge.resize(n+1);
        scc_id.resize(n+1);

        for (ll i = start_node; i <=
            end_node; i++){
            if(vis[i] == 0) dfs(i , 1 , edge
                , 1);
        }

        reverse_path(edge);

        vis.assign(n+1,0);
        ll id = 1;
        while(!finished.empty()){
            ll node = finished.top();
            finished.pop();
```

```

if(vis[node] == 0){
    number_of_scc++;
    dfs(node , id++ , reverse_edge ,
    2 );
}
}

scc_graph.resize(number_of_scc+1)
;
made_scc_graph(edge);
}

void dfs(ll node , ll id ,
vector<vector<ll>> &edge , ll
type)
{
    vis[node] = id;
    if(type == 2) scc_id[node] = id;

    for(auto child : edge[node]){
        if(vis[child] == 0) dfs(child ,
        id , edge , type);
    }

    if(type == 1 )
        finished.push(node);
    }

    void
    reverse_path(vector<vector<ll>>
    &edge){
        for (ll i = 0; i < edge.size();
        i++){
            for(auto child : edge[i])
                reverse_edge[child].push_back(i);
        }
    }

    void
    made_scc_graph(vector<vector<ll>>
    &edge){
        map<pair<ll,ll> , ll > check;

```

```

for (ll i = 1; i < edge.size();
i++)
{
    for(auto child : edge[i]){
        if(scc_id[i] != scc_id[child] &&
        check[{scc_id[i] ,
        scc_id[child]}] == 0){
            check[{scc_id[i],scc_id[child]}]
            = 1;

            scc_graph[scc_id[i]].push_back(sc
            c_id[child]); } } } } };

```

7. Topological Sort using BFS

Time complexity : $O(V+E)$

```

class Topological_BFS{
public :

    vector<ll> sorted;
    queue<ll> zero_inorder;
    Topological_BFS ( ll n , ll
    start_node , ll end_node ,
    vector<vector<ll>> &edge ,
    vector<ll> &inorder) {
        for (ll i = start_node; i <=
        end_node ; i++) {
            if(inorder[i] == 0)
                zero_inorder.push(i);
        }

        while(!zero_inorder.empty())
        {
            ll node = zero_inorder.front();
            zero_inorder.pop();
            sorted.push_back(node);

            for(auto child : edge[node]) {
                inorder[child]--;

```

```

if(inorder[child] == 0)
    zero_inorder.push(child);
}
} } };

```

8. Bridge And Articulation Point

Time complexity : $O(V+E)$

```

class BRIDGE{
public :

    vector<ll>
    vis,discover_time,low_time;
    vector<pair<ll,ll>> bridge_edge;
    set<ll> point;

    BRIDGE(ll n , vector<vector<ll>>
    &edge){
        vis.resize(n+2);
        discover_time.resize(n+2);
        low_time.resize(n+2);

        ll discover_id = 1;

        for (ll i = 0; i <= n; i++){
            if(vis[i] == 0) dfs(i,-
            1,discover_id,edge);
        }
    }

    void dfs(ll node , ll parent , ll
    &discover_id , vector<vector<ll>>
    &edge){
        vis[node] = 1;
        discover_time[node] =
        low_time[node] = discover_id++;

        ll count = 0;

```

```

for(auto child : edge[node]){
if(child == parent) continue;

if(vis[child] == 1){
low_time[node] =
min(low_time[node],discover_time[
child]);
continue;
}

dfs(child , node , discover_id ,
edge);
count++;

low_time[node] =
min(low_time[node],low_time[child
]);

if(discover_time[child] ==
low_time[child] )
bridge_edge.push_back({node,child
});

if(discover_time[node] <=
low_time[child] && parent != -1)
point.insert(node);

}

if(parent == -1 && count > 1)
{point.insert(node);}

};

```

9. Bipartite Check

Time complexity: $O(n)$

```

class Bipartite{
public:

```

```

vector<ll> save;
ll not_bipartite = 0;

Bipartite(vector<vector<ll>>
&edge){
save.resize(edge.size()+5,-1);
for(ll i = 0; i < edge.size();
i++) {
if(save[i] == -1) dfs(i , 0 ,
edge);
}
}

void dfs(ll node , ll color ,
vector<vector<ll>> &edge)
{
if(not_bipartite) return;
save[node] = color;

for(auto child : edge[node]){
if(save[child] == -1) dfs(child ,
!color , edge );
else if(save[child] == color)
{not_bipartite = 1; return;}
}
};

```

10. Tree

Centroid(single version)

Time complexity: $O(n)$

```

class tree_centroid{
public:

vector<ll> subtree_size;
vector<ll> mx_child;

```

```

ll centroid = 1; // set root
value
tree_centroid(ll n ,
vector<vector<ll>> &edge ){
if(n == 1) return;

subtree_size.resize(edge.size(),1
);
mx_child.resize(edge.size());

dfs(1,-1,edge);
centroid = 1; // set root value
ll mx_size = n/2;

while(1){
if( subtree_size[
mx_child[centroid]] > mx_size)
centroid = mx_child[centroid];
else break; } }

void dfs( ll node , ll parent ,
vector<vector<ll>> &edge) {
ll mx = 0;
for(auto child : edge[node]){
if(child == parent) continue;
dfs(child , node , edge);
subtree_size[node] +=
subtree_size[child];

if(mx < subtree_size[child]) {
mx_child[node] = child;
mx = subtree_size[child];
} } }

};

```

10. Floyd Warshall

```
// Time Complexity : O(n^3)
// Space Complexity : O(n^2)

// Declaration : Floyd_Warshall
f(n); then take all given edge
values then call run function
// Run : f.run(n);

class Floyd_Warshall{
public :
    vector<vector<ll>> dis;
    Floyd_Warshall(ll n ){
        dis.resize(n+2 , vector<ll>(n+2,
        LLONG_MAX));
        for(ll i = 1; i <= n; i++){
            dis[i][i] = 0;
        }
        void run(ll n ){
            for (ll k = 1; k <= n; k++){
                for (ll i = 1; i <= n; i++){
                    for(ll j = 1; j <= n; j++){
                        if(dis[i][k] == LLONG_MAX ||
                        dis[k][j] == LLONG_MAX) continue;
                        dis[i][j] = min(dis[i][j] ,
                        dis[i][k] + dis[k][j]);
                    }
                }
            }
        }
    };
};
```

11. cycle in directed Graph

```
class cycle{
public :
    vector<ll> vis;
    ll loop = 0;
```

```
cycle(ll n , vector<vector<ll>>
&adj){
    vis.resize(n+1);
    for (ll i = 0; i <= n; i++){
        {
            if(vis[i] == 0) dfs( i , adj);
        }
    }
    void dfs(ll node ,
    vector<vector<ll>> &adj)
    {
        if(vis[node] == 1){ loop = 1;
        return; }

        vis[node] = 1;

        for(auto child : adj[node])
        {
            if(vis[child] != 2) dfs(child ,
            adj);
            if(loop == 1) return;
        }
        vis[node] = 2;
    };
};
```

Number Theory

1. All Possible Subset

Time complexity: $O(2^n)$

```
void subset(vector<ll>&v ,
vector<ll> &result , ll index ){
    if(index == v.size()){
        for(auto u:result) cout<<u<< " "
        ;
        cout << endl;
        return;
    }
```

```
    }
    subset( v , result , index +
    1);
    result.push_back(v[index]);
    subset( v , result , index +
    1);

    result.pop_back();
}
```

2.Binary Exponentiation

Time complexity: $O(\log(b))$

```
ll power(ll a, ll b) {
    ll result = 1;
    while (b > 0) {
        if (b & 1) result *= a;
        a *= a;
        b >>= 1;
    }
    return result;
}
```

3.Bigmod

Time complexity: $O(\log(b))$

```
ll bigmod(ll a, ll b) {
    ll result = 1;
    while (b > 0) {
        if (b & 1) result =
        (result*a)%mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return result;
}
```

4. nCr

Time complexity: $O(2^n)$

```

class ncr{
public :

vector<ll> factorial;
ll mod = 1e9 + 7;

ncr( ll size){
factorial.resize(size + 1);
factorial[0] = 1;

ll val = 1;
for (ll i = 1; i <= size; i++){
val = (val * i) % mod;
factorial[i] = val;
}
}

// Here Add The Bigmod
Template

ll cal( ll n , ll r){
ll a = factorial[n];
ll b = (factorial[r] *
factorial[n - r]) % mod;
return (a*bigmod(b, mod-2) ) %
mod;
}
};

```

5. Modular inverse

Time complexity: $O(\log(n))$

```

class Modular_Inverse {
public :

ll mod;

Modular_Inverse(ll mod_value){
mod = mod_value;

```

```

}

// Here Add The Bigmod
Template

ll cal ( ll a)
{
return bigmod(a, mod - 2);
}
};

```

6. Inclusion-Exclusion

Time Complexity : $O(2^n)$

```

// How many numbers  $\leq n$  are
divisible by at least one element
of the array
LCM = 1 , index = 0 , size = 0

void Inclusion_Exclusion(ll n ,
vector<ll> &v , ll index ,ll size
, ll lcm , ll &ans){
if(index == v.size())
{
if(size == 0) return;
if(size % 2 == 0) ans -= (n/lcm);
else ans += (n/lcm);
return;
}

```

```

Inclusion_Exclusion(n , v ,
index+1 , size , lcm , ans);

```

```

ll x = (lcm * v[index]) /
__gcd(lcm , v[index]);
if(x <= n) Inclusion_Exclusion(n
, v , index+1 , size+1 , x , ans
);

}

```

7. SOD + NOD + Euler Totient

Just pass the Range value to the constructor when declaring the object and then the function will be work in that range

```

class Prime_Factor{
public :

```

```

vector<ll> min_factor;

```

```

Prime_Factor(ll range){
min_factor.resize(range);

```

```

min_factor[1] = 1;
for (ll i = 2; i < range; i++){
if(min_factor[i] == 0){
min_factor[i] = i;
for (ll j = i * i; j < range;
j+=i){
if(min_factor[j] == 0)
min_factor[j] = i;
} } } }

```

```

// Here add Binary Exponentiation
template

```

```

unordered_map<ll,ll>get_factors(1
ll n){
unordered_map <ll,ll> factor;
while(min_factor[n] != n ){
factor[min_factor[n]]++;
n /= min_factor[n];
}
if(n > 1) factor[n]++;
return factor;
}

```

```

11 Number_of_Divisors( ll n){
unordered_map<ll,ll> factor =
get_factors(n);
ll ans = 1;
for(auto u : factor) ans *=
(u.second + 1);
return ans;
}

11 Sum_of_Divisor( ll n){
unordered_map<ll,ll> factor =
get_factors(n);
ll ans = 1;
for(auto u : factor){
ans *= (power(u.first,u.second+1)
- 1) / (u.first - 1);
}
return ans;
}

11 Euler_Totient (ll n ){
unordered_map<ll,ll> factor =
get_factors(n);
ll ans = n;
for(auto u : factor) ans = (ans *
(u.first - 1)) / u.first;
return ans;
}

};

```

8. Segmented Sieve

Time Complexity :
 // constructor : $O(n \log \log n)$
 // range: $O((r - l + 1) * \log \log(r))$

```

class Segmented_seive{
public :

```

```

vector<ll> prime;
vector<bool> seive;

Segmented_seive(ll size) // size =
Sqrt(max_size)
{
seive.resize(size+10);
seive[0] = 1; seive[1] = 1;
for (ll i = 2; i <= size; i++){
if(seive[i] == 0){
prime.push_back(i);
for (ll j = i*i; j <= size; j+=i)
seive[j] = 1;
} } }

11 range(ll l, ll r){
ll size = r-l+1;
vector<bool> mark(size);
if(l == 1) mark[0] = 1;

for(auto i : prime){
if(i * i > r) break;
ll start = max(i * i, ((l + i - 1) /
i) * i);
for (ll j = start ; j <= r; j += i)
mark[j-l] = 1;
}

ll count = 0;
for(auto u : mark) if(u == 0)
count++;
return count;
}

};

```

10.String Arithmetic

Time complexity: $O(\log(b))$

```

class String_Arithmetic{
public :

11 modulo(string a , ll b){

```

```

ll val = 0;
for(auto c:a){
if( c == '-') continue;
val = (val * 10 + (c - '0')) % b;
}
return val;
}

void sum(string &a , string &b){
if(a.size() < b.size()) swap(a,b);
ll carry = 0;
ll n = a.size() , m = b.size();

for (ll i = 0; i < n; i++){
ll x = (a[n-1-i] - '0');
ll y = (i < m)? (b[m - 1 - i] - '0')
: 0;
x = x + y + carry;
a[n-1-i] = (x % 10) + '0';
carry = x / 10;
}
if(carry > 0) a = to_string(carry) +
a;
}

void mul_si(string &a , ll b){
ll carry = 0;
ll n = a.size();

for (ll i = n-1; i >= 0 ; i--){
ll x = (b * (a[i] - '0')) + carry;
a[i] = (x % 10) + '0';
carry = x / 10;
}
if(carry > 0) a = to_string(carry) +
a;
}
};

11. Bell + Ordered
Bell

```

// Bell Number

```
// Stirling Number Give us Number of
ways we can group/split n numbers
into k groups
```

```
// Number of way we can group/split
n numbers of elements
```

```
class Bell{
public :
vector<vector<ll>> s; // Stirling
Number
ll mod = 1e9+7;

Bell( ll range ){
s.resize(range , vector<ll>(range ,
-1));
}

ll stirling( ll n , ll k) // Rule :
s(n,k) = k * s(n-1,k) + s(n-1,k-1)
{
if(n == k || k == 1) return 1;
if(n == 0 || k <= 0 || k > n )
return 0;

if(s[n][k] != -1) return s[n][k];
return s[n][k] = ( ( k * stirling(n-
1 , k) ) % mod + stirling(n-1 , k-
1)) % mod;
}

ll cal( ll n ) // This calculate
Bell Number || Rule : sum of all
Stirling Number (1 to n)
{
ll ans = 0;
for (ll i = 1; i <= n; i++) ans =
(ans + stirling(n , i)) % mod;
return ans;
}
};

// Fubini Number || Ordered Bell
Number
```

```
// This Also make group/split of n
elements + Rank Each Group All
possible ways (No Two Group Can tie)
class Fubini {
public :
```

```
vector<vector<ll>> s; // Stirling
Number
vector<ll> factorial;
ll mod = 1e9+7;
```

```
Fubini(ll range){
s.resize(range+2 ,
vector<ll>(range+2 , -1));
factorial.resize(range+2 , 1);

for (ll i = 1; i < range+1; i++){
factorial[i] = (factorial[i-1] * i)
% mod;
}
}
```

```
ll stirling( ll n , ll k) // Rule :
s(n,k) = k * s(n-1,k) + s(n-1,k-1)
{
if(n == k || k == 1) return 1;
if(n == 0 || k <= 0 || k > n )
return 0;
```

```
if(s[n][k] != -1) return s[n][k];
return s[n][k] = ( ( k * stirling(n-
1 , k) ) % mod + stirling(n-1 , k-
1)) % mod;
}
```

```
// Here we take k! Because of each
group Ranking(permutation)
ll cal( ll n) // Rule : Sum of All
(Stirling Nuber * k!) in (1 to n)
{
ll ans = 0;
```

```
for (ll i = 1; i <= n; i++){
ans = (ans + (stirling(n,i) *
factorial[i]) % mod ) % mod;
}
```

```
return ans;
}
};
```

12. Derangement

A derangement is a permutation where no element stays in its original position.
n = number of element that need to be derange

```
ll derangement(ll n) {
if (n == 0) return 1;
if (n == 1) return 0;
if (dp[n] != -1) return
dp[n];
ll ans = (derangement(n-1) +
derangement(n-2)) % M;
ans = ((n-1) * ans) % M;
return dp[n] = ans;
}
```

13.Counting Coprime Pairs

```
class CoprimePairs {
public:
ll n;
vector<ll> arr;
vector<ll> cnt, f, mobb;
ll N;
```

```
CoprimePairs(ll _n, vector<ll>
&_arr, ll max_val = 1e6+5) {
n = _n;
arr = _arr;
N = max_val;
cnt.assign(N, 0);
f.assign(N, 0);
mobb.assign(N, 0);
preprocess(); }
```

```
void preprocess() {
```

```

for (auto &x : arr) cnt[x]++;
cnt[1] = n;

for (ll i = 2; i < N; i++)
for (ll j = i*2; j < N; j += i)
cnt[i] += cnt[j];

for (ll i = 1; i < N; i++)
f[i] = cnt[i] * (cnt[i]-1) / 2;

mobb[1] = 1;
for (ll i = 1; i < N; i++)
for (ll j = i*2; j < N; j += i)
mobb[j] -= mobb[i];
}

ll countCoprimePairs() {
ll ans = 0;
for (ll i = 1; i < N; i++)
ans += f[i] * mobb[i];
return ans;
}

ll countCoprimePairsDP() {
for (ll i = N-1; i >= 1; i--)
for (ll j = i*2; j < N; j += i)
f[i] -= f[j];
return f[1];
}
};

```

Data Structure

1. Ordered Set

```

// Time Complexcity : O(n log n)
for every oparetion

#include
<ext/pb_ds/assoc_container.hpp>
#include
<ext/pb_ds/tree_policy.hpp>

```

```

using namespace __gnu_pbds;

template<typename T>
using ordered_set = tree<
T,
null_type, // use mapped_type
here if map-like behavior needed
less<T>, // comparator
(greater<T> for descending order)
rb_tree_tag, // underlying tree
structure
tree_order_statistics_node_update
>;

// ordered_set<ll> o_set;
Declaration
// s.insert(x); Insert
// s.erase(x); Remove
// s.order_of_key(x);
count element smaller than x
// *s.find_by_order(k); Get k-th
element(index)

```

2.Euler Tour

Time complexity: O(n)

// When use this with segment tree Remember -> For update you need to update in two position (in + out) in segment tree

```

class euler_tour{
public:

vector<ll> in,out,flat_tree;

euler_tour(vector<vector<ll>>
&edge , vector<ll> &value){
in.resize(edge.size(),-1);
out.resize(edge.size(),-1);

```

```

flat_tree.resize(2*edge.size()-
2);

```

```

ll pos = 0;
dfs(1 , edge , pos , -1);

for(ll i = 1; i < edge.size();
i++) {
flat_tree[in[i]] = value[i-1];
flat_tree[out[i]] = value[i-1];
}
}

```

```

void dfs(ll node ,
vector<vector<ll>> &edge , ll
&pos , ll parent){
in[node] = pos++;
for(auto child : edge[node]){
if( child == parent) continue;
dfs(child , edge , pos , node);
}
out[node] = pos++;
}
};

```

3.Segment Tree

```

class segment_tree{

public :

vector<ll> save, lazy;

segment_tree(ll size){
save.resize(4*size);
lazy.resize(4*size);
}

```

```

void push(ll node, ll start, ll end){
    if (lazy[node] == 0) return;

    // apply lazy to current node
    save[node] += (end - start + 1) * lazy[node];

    if (start != end) {
        lazy[node*2] += lazy[node];
        lazy[node*2 + 1] += lazy[node];
    }
    lazy[node] = 0;
}

void create(ll node, ll start, ll end, vector<ll> &v){
    if (start == end) {
        save[node] = v[start]; // set leaf
        return;
    }

    ll mid = (start + end) / 2;
    create(node*2, start, mid, v);
    create(node*2 + 1, mid+1, end, v);

    save[node] = save[node*2] + save[node*2 + 1]; // merge up
}

void update(ll node, ll start, ll end, ll l, ll r, ll value){
    push(node, start, end); // push before using

    if (start > r || end < l) return;

    if (l <= start && end <= r){

```

```

        lazy[node] += value; // assign lazy
        push(node, start, end); // apply immediately
        return;
    }

    ll mid = (start + end) / 2;
    update(node*2, start, mid, l, r, value);
    update(node*2 + 1, mid+1, end, l, r, value);

    save[node] = save[node*2] + save[node*2 + 1]; // merge up
}

void query(ll node, ll start, ll end, ll l, ll r, ll carry, ll &ans){
    push(node, start, end); // lazy correction

    if (start > r || end < l) return;

    if (l <= start && end <= r) {
        ans += save[node]; // add segment contribution
        return;
    }

    ll mid = (start + end) / 2;
    query(node*2, start, mid, l, r, carry, ans);
    query(node*2 + 1, mid+1, end, l, r, carry, ans);
}

};

```

4.Mo's Algorithm(Static Range Query)

```

// Time Complexity:
// sorting query: O(Q * log Q)
// Total : O((N + Q) * sqrt(N))

ll block; // Block size for mo's algorithm
struct query // structure to hold each query
{
    ll l,r,index;
};

bool compare(const query &a, const query &b) // custom compare function for sorting
{
    if(a.l/block == b.l/block)
    {
        return a.r < b.r;
    }
    return a.l/block < b.l/block;
}

void mo_sort(vector<query> &q , vector<ll> &v){
    block = sqrt(v.size()) + 1;
    sort(q.begin() , q.end() , compare);
}

```

5.Heavy Light Decomposition(HLD) +

Segment Tree (for Path Query)

// Here add segment tree
template with 1 based index

// Here edge is the tree edge and
value is the value for each node in
tree

```
class HLD{
public:
    vector<ll>
    depth,parent,heavy,head,pos,subtree_
    size , flat_tree;
    segment_tree segment;
```

```
HLD(vector<vector<ll>> &edge ,
vector<ll> &value ) :
    segment(edge.size()){
    depth.resize(edge.size());
    parent.resize(edge.size());
    heavy.resize(edge.size(),-1);
    head.resize(edge.size());
    pos.resize(edge.size());
```

```
    subtree_size.resize(edge.size(),1);
    flat_tree.resize(edge.size());
```

```
    dfs1(1,edge,0,1);
    ll count = 1;
    dfs2(1,edge , count , -1);
```

```
    for (ll i = 1; i < edge.size(); i++)
        flat_tree[pos[i]] = value[i-1];
```

```
    segment.create(1 , 1 ,
    flat_tree.size()-1 , flat_tree);
}
```

```
void dfs1(ll node ,
vector<vector<ll>> &edge , ll height
, ll par){
    depth[node] = height;
    parent[node] = par;
```

```
    ll mx_subtree = 0;
    for(auto u : edge[node]){
        if(u == par) continue;
```

```
    dfs1(u,edge,height+1,node);
```

```
    subtree_size[node] +=
    subtree_size[u];
    if(subtree_size[u] > mx_subtree)
    {mx_subtree = subtree_size[u];
    heavy[node] = u;}
```

```
    }
}
```

```
void dfs2(ll node ,
vector<vector<ll>> &edge , ll &count
, ll top ){
    if(top == -1) top = node;
    head[node] = top;
```

```
    pos[node] = count++;
    if(heavy[node] == -1) return;
```

```
    dfs2(heavy[node] , edge , count ,
    head[node]);
```

```
    for(auto u : edge[node]){
        if(u == heavy[node] || u ==
        parent[node]) continue;
        dfs2(u,edge,count , -1);
    }
}
```

```
ll query_path(ll u , ll v )
{
    ll ans = LLONG_MIN; // This Change
    based on the problem
```

```
    while(head[u] != head[v]){
        if(depth[head[u]] > depth[head[v]]){
```

```
        segment.query(1 , 1 ,
        flat_tree.size()-1 , pos[head[u]] ,
        pos[u] , 0 , ans);
        u = parent[head[u]];
    }
    else{
        segment.query(1 , 1 ,
        flat_tree.size()-1 , pos[head[v]] ,
        pos[v] , 0 , ans);
        v = parent[head[v]];
    }
}
```

```
    segment.query(1 , 1 ,
    flat_tree.size()-1 ,
    min(pos[u],pos[v]) ,
    max(pos[u],pos[v]) , 0 , ans);
```

```
    return ans;
}
```

```
void update_path(ll u , ll v , ll
value){
    segment.update(1 , 1 ,
    flat_tree.size()-1 , pos[u] , pos[u]
    , value);
}
```

```
};
```

// This is 1 based index HLD for
each array segment tree should also
be 1 based index

6. Trie

```
class Trie {
public:
```

```
    struct Node {
        bool endMark;
        array<Node*, 26> next;
```

```

Node() {
endMark = false;
next.fill(nullptr);
}
};

Node* root;

Trie() {
root = new Node();
}

void insert(const string &s) {
Node* cur = root;
for (char c : s) {
ll id = c - 'a';
if (cur->next[id] == nullptr) {
cur->next[id] = new Node();
}
cur = cur->next[id];
}
cur->endMark = true;
}

bool search(const string &s) {
Node* cur = root;
for (char c : s) {
ll id = c - 'a';
if (cur->next[id] == nullptr)
return false;
cur = cur->next[id];
}
return cur->endMark;
}

bool startsWith(const string
&prefix){
Node* cur = root;
for (char c : prefix) {
ll id = c - 'a';
if (cur->next[id] == nullptr)
return false;

```

```

cur = cur->next[id];
}
return true;
}
};

```

String

1. String Hashing

Time Complexity: $O(n)$

// this convert string into a ll hash value in $O(n)$

// here you need to declare prime and mod value for use

```

ll get_hash(string &s){
ll val = 0;
for (ll i = 0; i < s.size(); i++){
val = ( (val * prime) + (ll)s[i] ) % mod;
}
return val;
}

```

2. Z Algorithm

```

vector<ll> z_function(string
s) {
ll n = s.size();
vector<ll> z(n);
for (ll i = 1, l = 0, r = 0; i
< n; i++) {

```

```

if (i <= r) z[i] = min(z[i -
l], r-i+1);
while (i + z[i] < n and
s[z[i]] == s[i + z[i]])
z[i]++;
if (i + z[i] - 1 > r) {
l = i;
r = i + z[i] - 1;
}
}
return z;
}

```

Others

1. Maximum Subarray sum

```

ll max_subarray_sum(vector<ll> &v
){
ll sum = 0;
ll mx = LLONG_MIN;

for (ll i = 0; i < v.size(); i++){
sum += v[i];
mx = max(mx, sum);
if (sum < 0) sum = 0;
}

return mx;
}

```

2. Ternary Search

```

double ternary_search(double l,
double r) {
double eps = 1e-9; //set the error
limit

```

```

while (r - l > eps) {
double m1 = l + (r - l) / 3;
double m2 = r - (r - l) / 3;
double f1 = f(m1);
double f2 = f(m2);

if (f1 < f2) l = m1;
else r = m2;
}
return f(l);
}

```

Equation

Area of trapezium: $0.5 * h * (a+b)$

Sum of odd number formula:
 n^2

Sum of even number formula:
 $n * (n+1)$

Sum of squares formula: $\frac{1}{6} * n * (n+1) * (2n+1)$

Sum of cube series formula:
 $\frac{1}{4} * n^2 * (n+1)^2$

Series Sum: $n/2 * (2a + (n-1)*d)$

Number of lattice point
between two points: =
 $\gcd(|x1-x2|, |y1-y2|) + 1$

Important Pythagorean
Triples:

3, 4, 5
5, 12, 13
7, 24, 25
8, 15, 17
9, 40, 41
20, 21, 29

Extra

1. Big Prime Number

- 970592641 (31-bit number)
- 31443539979727 (45-bit)
- 3006703054056749 (52-bit)

2. Prime Count

- 5133 upto 5e4
- 9592 upto 1e5
- 17984 upto 2e5
- 78498 upto 1e6
- 5761455 upto 1e8

3. Max NOD

- 100 for $n = 5e4$
- 500 for $n = 1e7$
- 2000 for $n = 1e10$
- 200 000 for $n = 1e19$

4. Max Unique Prime Factors

- 6 upto 5e5
- 7 upto 9e6
- 8 upto 2e8
- 9 upto 6e9
- 11 upto 7e12
- 15 upto 3e19

Combination And permutation

$$P(n, r) = nPr = \frac{n!}{(n-r)!}$$

if we have repeted values

$$\text{Arrangements} = \frac{n!}{n_1! \cdot n_2! \cdot \dots \cdot n_k!}$$

$$C(n, r) = nCr = \frac{n!}{r! \cdot (n-r)!}$$

$$\sum_{r=0}^n C(n, r) = 2^n$$

Expected value :

$$E[X] = \sum_i x_i \cdot p_i$$

- $\sum_{0 \leq k \leq n} \binom{n-k}{k} = \text{Fib}_{n+1}$
- $\binom{n}{k} = \binom{n}{n-k}$
- $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
- $k \binom{n}{k} = n \binom{n-1}{k-1}$
- $\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$
- $\sum_{i=0}^n \binom{n}{i} = 2^n$
- $\sum_{i \geq 0} \binom{n}{2i} = 2^{n-1}$
- $\sum_{i \geq 0} \binom{n}{2i+1} = 2^{n-1}$
- $\sum_{i=0}^k (-1)^i \binom{n}{i} = (-1)^k \binom{n-1}{k}$
- $\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k}$
- $\sum_{i=0}^k \binom{n+i}{n} = \binom{n+k+1}{k}$
- $\sum_{i=0}^k \binom{i}{n} = \binom{k+1}{n+1}$
- $1 \cdot \binom{n}{1} + 2 \cdot \binom{n}{2} + 3 \cdot \binom{n}{3} + \dots$

Function	Description
<code>sin(x)</code>	Sine of x
<code>cos(x)</code>	Cosine of x
<code>tan(x)</code>	Tangent of x
<code>asin(x)</code>	Arc Sine (Inverse Sin)
<code>acos(x)</code>	Arc Cosine (Inverse Cos)
<code>atan(x)</code>	Arc Tangent (Inverse Tan)
<code>atan2(y, x)</code>	Arc Tangent with 2 args*
<code>M_PI</code>	Default pi value

Geometry

Circle Equation :

$$x^2 + y^2 + 2gx + 2fy + c = 0$$

$$\text{Center} = (-g, -f)$$

$$\text{Radius} = \sqrt{g^2 + f^2 - c}$$

Circle with center (h,k)

$$(x - h)^2 + (y - k)^2 = r^2$$

Length of Tangent from External Point

If distance from center = d:

$$\text{Tangent length} = \sqrt{d^2 - r^2}$$

Or from point (x_1, y_1) :

$$\ell = \sqrt{x_1^2 + y_1^2 - r^2}$$

Equation of Tangent at a Point

For $x^2 + y^2 = r^2$, tangent
at (x_1, y_1) :

$$xx_1 + yy_1 = r^2$$

Family of Circles

Through Two Points

(Important in Olympiad)

Through points (x_1, y_1) and (x_2, y_2) :

$$S_1 + \lambda S_2 = 0$$

where S_1, S_2 are circle equations through those points.

Chord Length Formula :

$$L = 2\sqrt{r^2 - d^2}$$

Arc Length : $L = r\theta$

Sector Area: $A =$

$$(1/2)r^2\theta$$

Distance Between Two Circle Centers :

$$d = \sqrt{(h_1 - h_2)^2 + (k_1 - k_2)^2}$$

Triangle

Area : $A = \sqrt{s(s - a)(s - b)(s - c)}$

Half Perimeter: $s = (a + b + c)/2$

law of sine : $a/\sin A = b/\sin B = c/\sin C$

Law of Cosines : $a^2 = b^2 + c^2 - 2bc \cos A$

Inradius & Circumradius :

$$r = A/s$$

$$R = abc / \sqrt{(a+b+c)(a+b-c)(b+c-a)(c+a-b)}$$

$$R = a / (2 \sin A)$$

Coordinate Geometry - Area :

$$A = (1/2) |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$$

Code Template

```
#include
<bits/stdc++.h>
using namespace std;
#define ll long long
#define endl "\n"
int main()
{
    ios_base::sync_with_st
    dio(false); cin.tie(NUL
    L);

    ll int t; cin >> t;
    while(t--) {

    }
}
```

Stress Testing Bash Code :

```
#!/bin/bash

# Compile the programs
g++ my_solution.cpp -o
my_solution -O2 -std=c++17 ||
{ echo "Compilation failed for
my_solution.cpp"; exit 1; }
g++ correct_solution.cpp -o
correct_solution -O2 -
std=c++17 || { echo
"Compilation failed for
correct_solution.cpp"; exit 1;
}
g++ Random_Test_Case.cpp -o
Random_Test_Case -O2 -
std=c++17 || { echo
"Compilation failed for
Random_Test_Case.cpp"; exit 1;
}

# Stress test loop
for ((i = 1; ; ++i)); do
    ./Random_Test_Case $i >
random_input
    ./my_solution <
random_input > my_output
    ./correct_solution <
random_input > correct_output

    if ! diff -w my_output
correct_output > /dev/null;
then
        echo -e "\nTest failed
on iteration: $i"
```

```
        echo "Seed used: $i"
        echo -e "\nWA on the
following test:"
        cat random_input
        echo -e "\nYour answer
is:"

        cat my_output
        echo -e "\nCorrect
answer is:"
        cat correct_output
        exit 1
    fi

    echo "Passed test: $i"
done

# Cleanup
trap "rm -f random_input
my_output correct_output" EXIT
```

File Names :

- 1) correct_solution.cpp
- 2) my_solution.cpp
- 3) Random_Test_Case.cpp
- 4) cheaker.sh

Random Test Case Generator :

```
// Function to generate a random
number in a given range [min,
max]
long long rand_range(long long
min_val, long long max_val) {
    static std::mt19937
generator(

std::chrono::steady_clock::now().
time_since_epoch().count()
);

std::uniform_int_distribution<lon
g long> distribution(min_val,
max_val);
    return
distribution(generator);
}
```